

数据库遇见新硬件

周烜

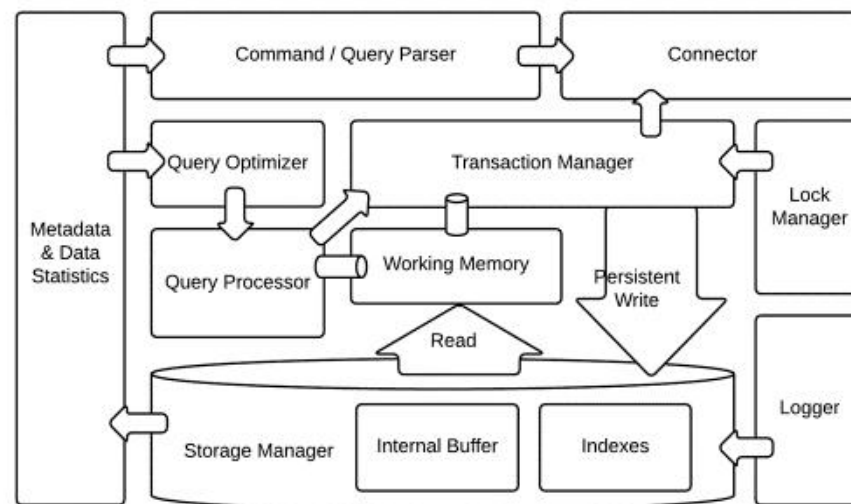
华东师范大学 数据学院



数据库系统是极复杂的基础软件

因此

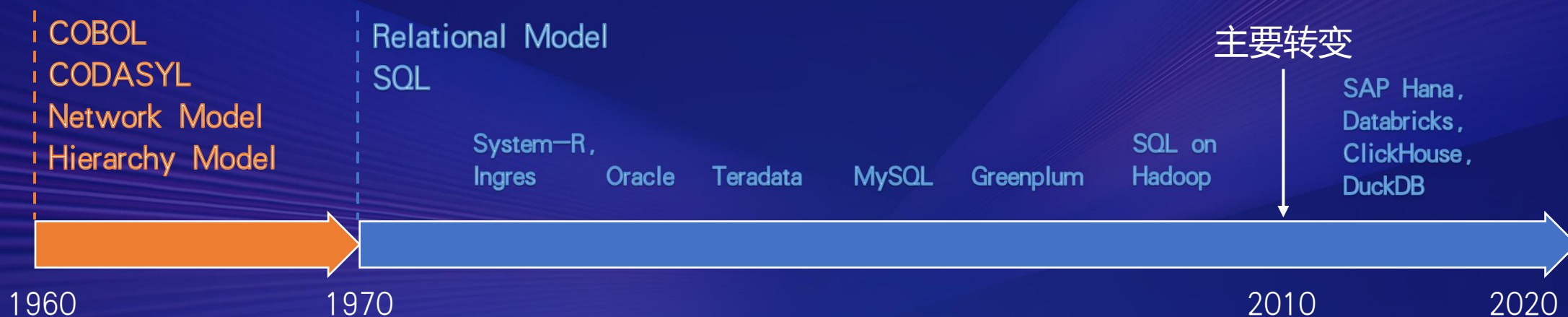
- 修改代价高
- 演进缓慢
- 对新技术异常挑剔



什么促使数据库系统形态变化？

- 应用需求的变化
 - 互联网应用、人工智能 → 负载增长、数据多样化
- 硬件平台的革新
 - 廉价服务器、云平台、**新型存储、新型处理器**
- 软件开发模式的转变
 - 面向对象、敏捷开发、微服务架构、事件驱动架构

回顾历史：硬件发展如何改变数据库系统形态？



性能瓶颈：I/O-Bounded → Half I/O-Bounded, Half CPU-Bounded

早期数据库性能被认为是I/O-dominant

A History and Evaluation of System R

Donald D. Chamberlin	Thomas G. Price
Morton M. Astrahan	Franco Putzolu
Michael W. Blasgen	Patricia Griffiths Selinger
James N. Gray	Mario Schkolnick
W. Frank King	Donald R. Slutz
Bruce G. Lindsay	Irving L. Traiger
Raymond Lorie	Bradford W. Wade
James W. Mehl	Robert A. Yost

IBM Research Laboratory
San Jose, California

1. Introduction

Throughout the history of information storage in computers, one of the most readily observable trends has been the focus on data independence. C.J. Date [27] defined data independence as "immunity of applications to change in storage structure and access strategy." Modern database systems offer data independence by providing a high-level user interface through which users deal with the information content of their data, rather than the various bits, pointers, arrays, lists, etc. which are used to represent that information. The system assumes responsibility for choosing an appropriate internal

representation for the information; indeed, the representation of a given fact may change over time without users being aware of the change.

The relational data model was proposed by E.F. Codd [22] in 1970 as the next logical step in the trend toward data independence. Codd observed that conventional database systems store information in two ways: (1) by the contents of records stored in the database, and (2) by the ways in which these records are connected together. Different systems use various names for the connections among records, such as links, sets, chains, parents, etc. For example, in Figure 1(a), the fact that supplier Acme supplies bolts is represented by connections between the relevant part and supplier records. In such a system, a user frames a question, such as "What is the lowest price for bolts?", by writing a program which "navigates" through the maze of connections until it arrives at the answer to the question. The user of a "navigational" system has the burden (or opportunity) to specify exactly how the query is to be processed; the user's algorithm is then embodied in a program which is dependent on the data structure that existed at the time the program was written.

Relational database systems, as proposed by Codd, have two important properties: (1) all information is

represented by connections between the relevant part and supplier records. In such a system, a user frames a question, such as "What is the lowest price for bolts?", by writing a program which "navigates" through the maze of connections until it arrives at the answer to the question. The user of a "navigational" system has the burden (or opportunity) to specify exactly how the query is to be processed; the user's algorithm is then embodied in a program which is dependent on the data structure that existed at the time the program was written.

Relational database systems, as proposed by Codd, have two important properties: (1) all information is

Communications
of
the ACM

October 1981
Volume 24
Number 10

same time, Phase Zero taught us a number of valuable lessons which greatly influenced the design of our later implementation. Some of these lessons are summarized below.

(1) The optimizer should take into account not just the cost of fetching tuples, but the costs of creating and manipulating TID lists, then fetching tuples, then fetching the data pointed to by the tuples. When these "hidden costs" are taken into account, it will be seen that the manipulation of TID lists is quite expensive, especially if the TID lists are managed in the database rather than in main storage.

(2) Rather than "number of tuples fetched," a better measure of cost would have been "number of I/Os." This improved cost measure would have revealed the great importance of clustering together related tuples on physical pages so that several related tuples could be fetched by a single I/O. Also, an I/O measure would have revealed a serious drawback of XRM: Storing

DATABASE SYSTEMS The Complete Book

Second Edition

Hector Garcia-Molina

Jeffrey D. Ullman

Jennifer Widom

Department of Computer Science
Stanford University

Dominance of I/O cost: The time taken to perform a disk access is much larger than the time likely to be used manipulating that data in main memory. Thus, the number of block accesses (*Disk I/O's*) is a good approximation to the time needed by the algorithm and should be minimized.

硬件发展是如何导致这一变化的？

1980s

I/O:

1MB/S



CPU:

2MIPS



2020s

10GB/S



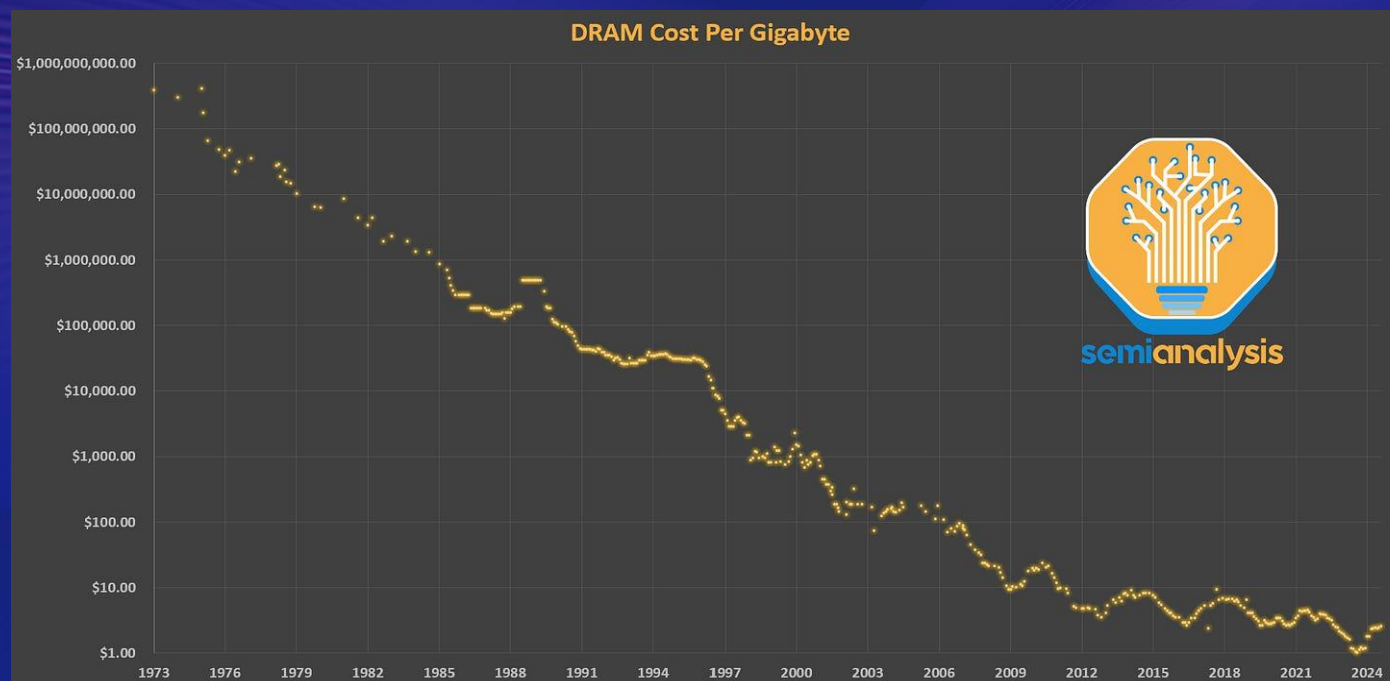
200KMIPS



实际上有更复杂和综合的情况！

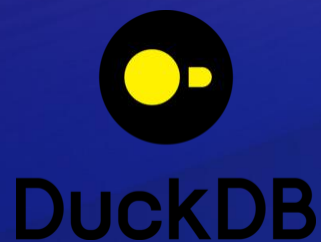
实际情况：内存容量增长

- 内存从80年代的128KB增长到现在的128GB
- 导致大量数据可以缓存在内存，避免了大量I/O操作



实际情况：大内存下的数据库查询引擎

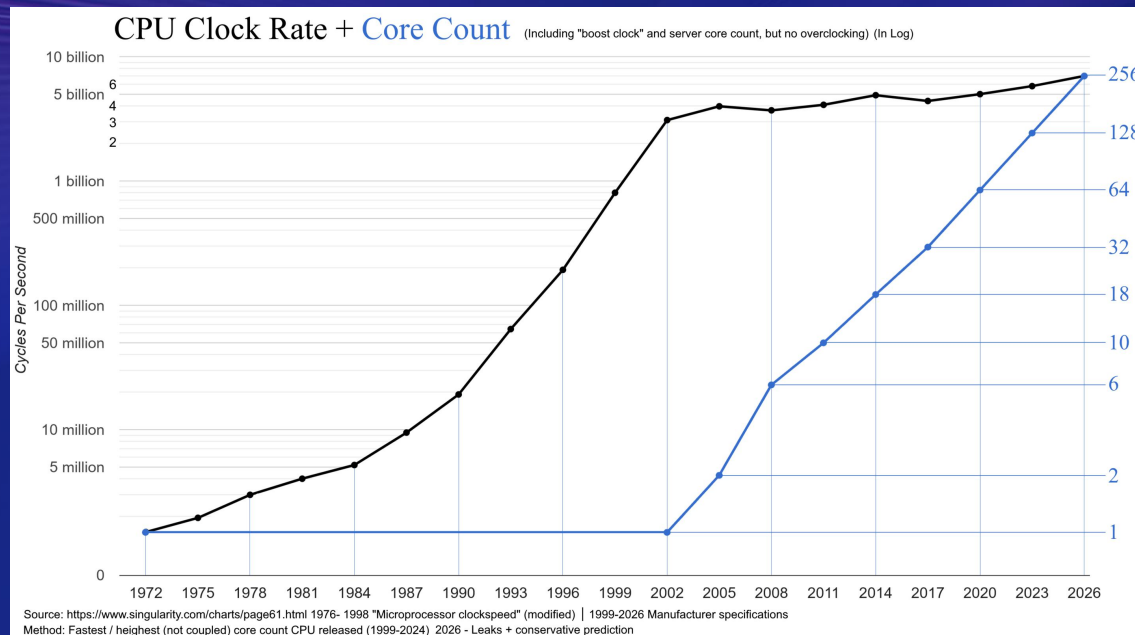
- 传统数据库：重量级算子（Join和Agg）使用外部算法
- 现代OLAP引擎：内存不足时才将中间结果暂存到外存（Spilling）



HASH GROUP BY Impact: High Memory Usage: HASH GROUP BY operations can consume a lot of memory, especially when dealing with large datasets or high cardinality groupings. This can lead to memory exhaustion and increased disk I/O if ClickHouse starts using temporary disk storage.**CPU Overhead:** Calculating hash values for each row can be CPU-intensive, particularly with complex grouping expressions.**Risk of Spilling to Disk:** If the memory gets exhausted, ClickHouse might spill data to disk, significantly slowing down the query.

实际情况：CPU串行性能已停止增长

- CPU时钟频率于2004年停止增长。从此，算力增长依赖并行化：
 - 向量化 (SIMD)、多核 (Multicore)
- 传统数据库未重视并行化，导致CPU瓶颈的出现



实际情况：并行化OLAP获得显著性能收益

- 2005年，CWI构建了首个向量化查询处理引擎，将性能提升2个数量级。

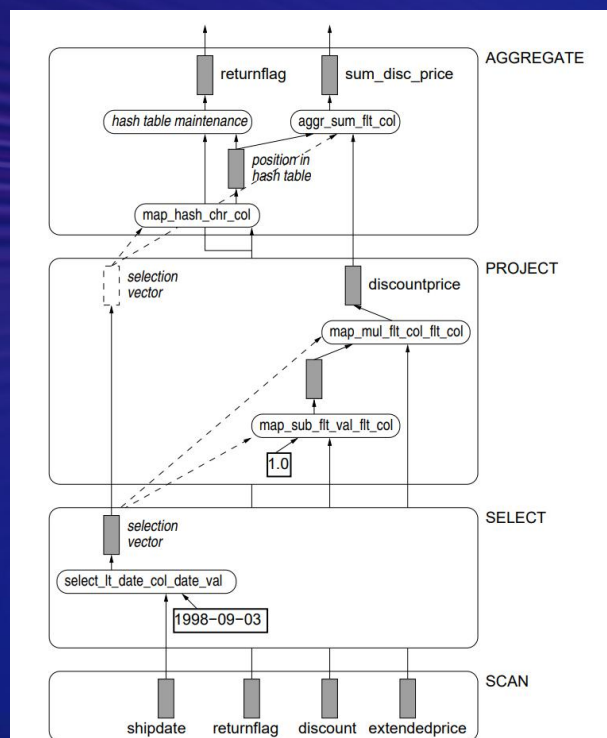


Figure 6: Execution scheme of a simplified TPC-H Query 1 in MonetDB/X100

Q	MonetDB/MIL	MonetDB/X100, 1CPU			DB2, 4CPU
	SF=1	SF=1	SF=1	SF=100	SF=100
1	3.72	0.50	0.31	30.25	229
2	0.46	0.01	0.01	0.81	19
3	2.52	0.04	0.02	3.77	16
4	1.56	0.05	0.02	1.15	14
5	2.72	0.08	0.04	11.02	72
6	2.24	0.09	0.02	1.44	12
7	3.26	0.22	0.22	29.47	81
8	2.23	0.06	0.03	2.78	65
9	6.78	0.44	0.44	71.24	274
10	4.40	0.22	0.19	30.73	47
11	0.43	0.03	0.02	1.66	20
12	3.73	0.09	0.04	3.68	19
13	11.42	1.26	1.04	148.22	343
14	1.03	0.02	0.02	2.64	14
15	1.39	0.09	0.04	14.36	30
16	2.25	0.21	0.14	15.77	64
17	2.30	0.02	0.02	1.75	77
18	5.20	0.15	0.11	10.37	600
19	12.46	0.05	0.05	4.47	81
20	2.75	0.08	0.05	2.45	35
21	8.85	0.29	0.17	17.61	428
22	3.07	0.07	0.04	2.30	93
		AthlonMP		Itanium2	

Table 4: TPC-H Performance (seconds)

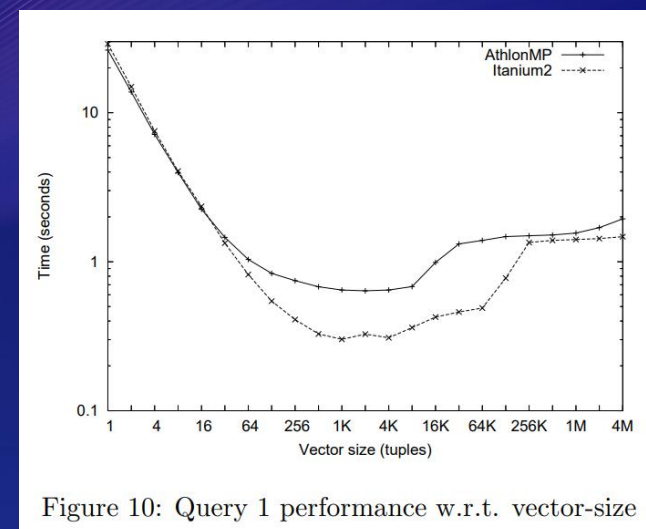


Figure 10: Query 1 performance w.r.t. vector-size

Peter A. Boncz, Marcin Zukowski,
 Niels Nes: MonetDB/X100: Hyper-
 Pipelining Query Execution. CIDR
 2005: 225–237

小结：多种因素的综合结果

	<i>1980s</i>	<i>2020s</i>	增长倍数
<i>I/O:</i>	1MB/S	10GB/S	10^4
<i>CPU:</i>	2MIPS	200KMIPS	10^5
<i>memory size:</i>	128KB	128GB	10^6
<i>Clock Rate:</i>	6MHZ	6GHZ	10^3

主要创新：内存OLAP技术

- 1999年，CWI开始构建纯内存的OLAP引擎MonetDB



Database Architecture Optimized for the new Bottleneck: Memory Access

Peter Boncz*
 Data Distilleries B.V.
 Amsterdam · The Netherlands
 P.Boncz@ddi.nl

Stefan Manegold Martin Kersten
 CWI
 Amsterdam · The Netherlands
 {S.Manegold,M.Kersten}@cwi.nl

Abstract

In the past decade, advances in speed of commodity CPUs have far out-paced advances in memory latency. Main-memory access is therefore increasingly a performance bottleneck for many computer applications, including database systems. In this article, we use a simple scan test to show the severe impact of this bottleneck. The insights gained are translated into guidelines for database architecture; in terms of both data structures and algorithms. We discuss how vertically fragmented data structures optimize cache performance on sequential data access. We then focus on equi-join, typically a random-access operation, and introduce radix algorithms for partitioned hash-join. The performance of these algorithms is quantified using a detailed analytical model that incorporates memory access cost. Experiments that validate this model were performed on the Monet database system. We obtained exact statistics on events like TLB misses, L1 and L2 cache misses, by using hardware performance counters found in modern CPUs. Using our cost model, we show how the carefully tuned memory access pattern of our radix algorithms make them perform well, which is confirmed by experimental results.

*This work was carried out when the author was at the University of Amsterdam, supported by SIDON grant 612-23-431. Permission to copy without fee all or part of this material is granted provided that the copies are not made or distributed for direct commercial advantage, the VLDB copyright notice and the title of the publication and its date appear, and notice is given that copying is by permission of the Very Large Data Base Endowment. To copy otherwise, or to republish, requires a fee and/or special permission from the Endowment.

Proceedings of the 25th VLDB Conference, Edinburgh, Scotland, 1999.

1 Introduction

Custom hardware – from workstations to PCs – has been experiencing tremendous improvements in the past decades. Unfortunately, this growth has not been equally distributed over all aspects of hardware performance and capacity. Figure 1 shows that the speed of commercial microprocessors has been increasing roughly 70% every year, while the speed of commodity DRAM has improved by little more than 50% over the past decade [Mow94]. Part of the reason for this is that there is a direct tradeoff between capacity and speed in DRAM chips, and the highest priority has been for increasing capacity. The result is that from the perspective of the processor, memory has been getting slower at a dramatic rate. This affects all computer systems, making it increasingly difficult to achieve high processor efficiencies.

Three aspects of memory performance are of interest: bandwidth, latency, and address translation. The only way to reduce effective memory latency for appli-

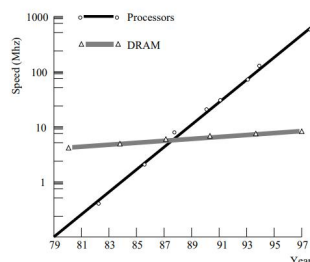


Figure 1: Hardware trends in DRAM and CPU speed

In memoriam Dik Winter

VLDB 10-year Best Paper Award for hardware-aware database technology

Researchers Peter Boncz, Stefan Manegold, and Martin Kersten of the Centrum Wiskunde & Informatica (CWI) in Amsterdam won the prestigious VLDB 10-year Best Paper Award on August 27.



主要创新：内存OLAP技术

- 1999年，CWI开始构建纯内存的OLAP引擎MonetDB
- 2005年，CWI利用向量化技术将MonetDB改进为Vectorwise
- 2010年，TU Munich构建HyPer，将JIT技术应用于OLAP
- 2016年，Clickhouse发布，采用了大量内存+CPU优化
- 2019年，CWI发布DuckDB，采用Push-Based Execution

Clickhouse : 工程细节决定性能

Attention to Low-Level Details

But many other database management systems use similar techniques. **What really makes ClickHouse stand out is attention to low-level details.** Most programming languages provide implementations for most common algorithms and data structures, but they tend to be too generic to be effective. Every task can be considered as a landscape with various characteristics, instead of just throwing in random implementation.

Tips for building your own high-performance software

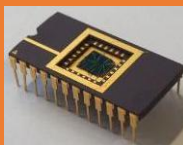
- Keep in mind low-level details when designing your system.
- Design based on hardware capabilities.
- Choose data structures and abstractions based on the needs of the task.
- Provide specializations for special cases.
- Try new, "best" algorithms, that you read about yesterday.
- Choose an algorithm in runtime based on statistics.
- Benchmark on real datasets.
- Test for performance regressions in CI.
- Measure and observe everything.

未来将如何？
What's next?

讨论三个硬件趋势

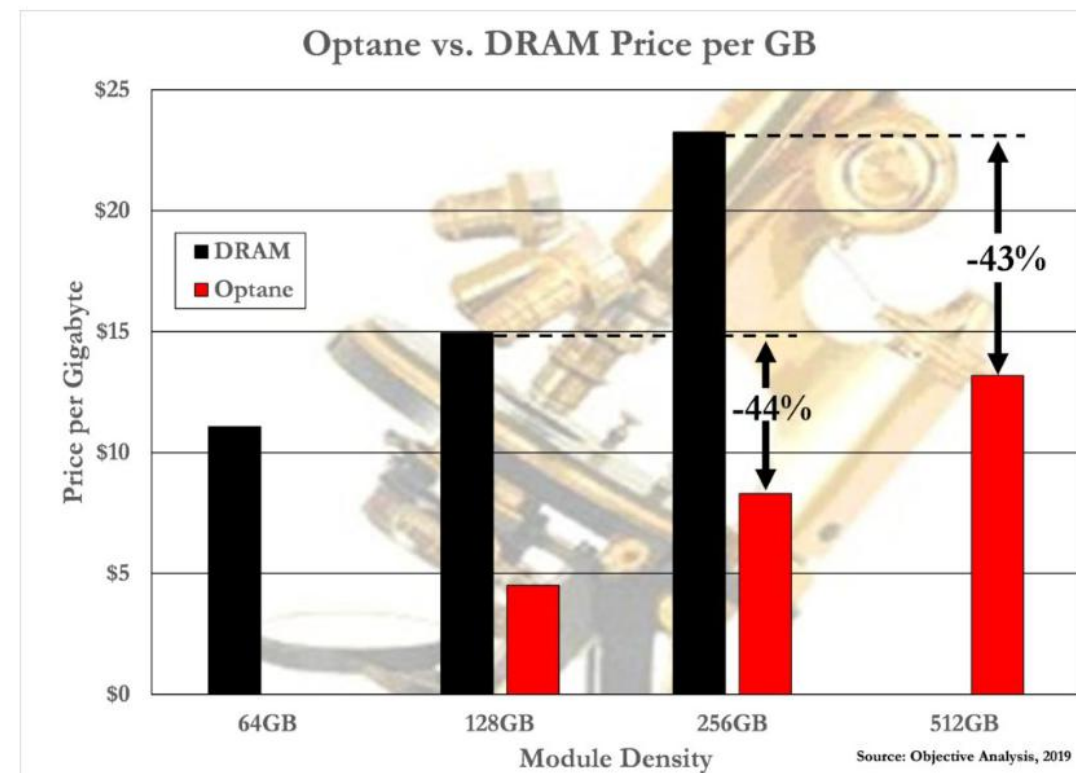
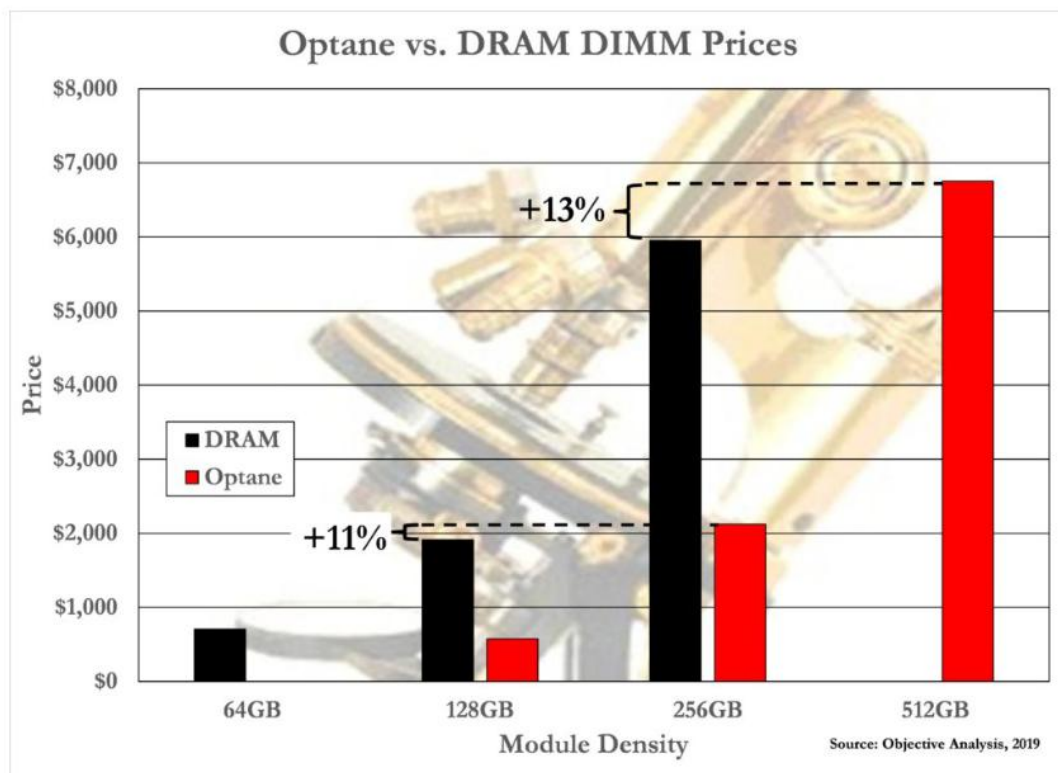
- 非易失内存
- GPU
- 专用硬件 (FPGA、ASIC)

非易失内存 / Non-Volatile Memory

Features of Different Storage Devices	 DRAM	 PCM	 STT-RAM	 ReRAM	 Intel AEP	 NAND SSD
Non-Volatility	✗	✓	✓	✓	✓	✓
Byte-Addressability	✓	✓	✓	✓	✓	✗
Read Latency (ns)	80	96	290	360	280	10^4
Write Latency (ns)	80	800	290	400	95	10^5
Write Endurance	10^{16}	10^9	10^{12}	10^{13}	$10^6 \sim 10^9$	10^5

Intel Optane DC Persistent Memory (AEP) 是最早商业化的产品

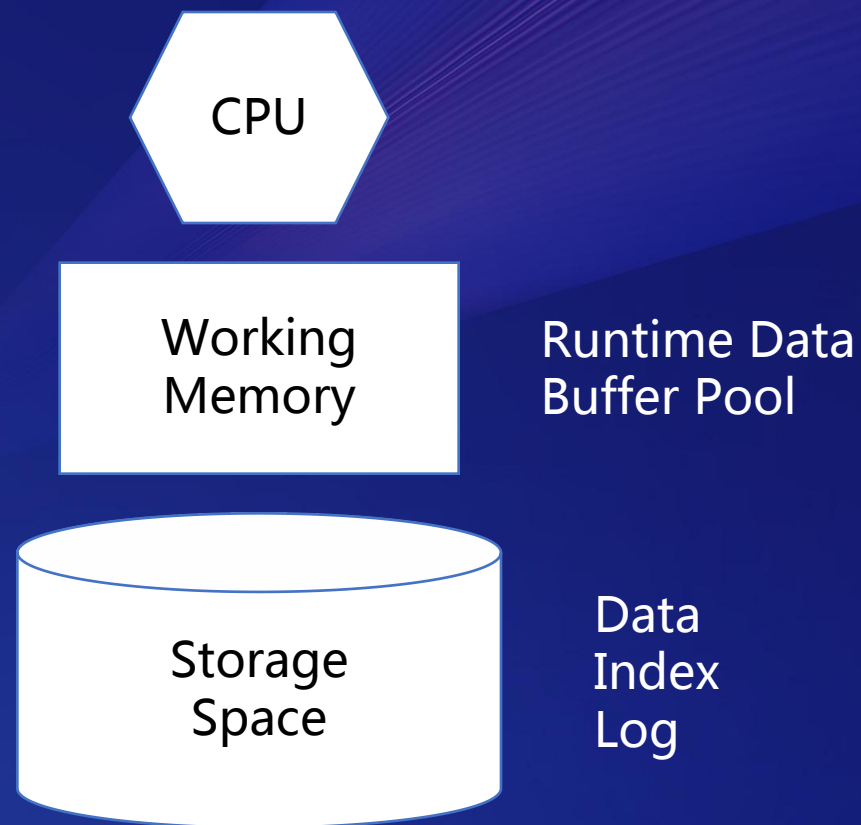
Intel AEP的性价比



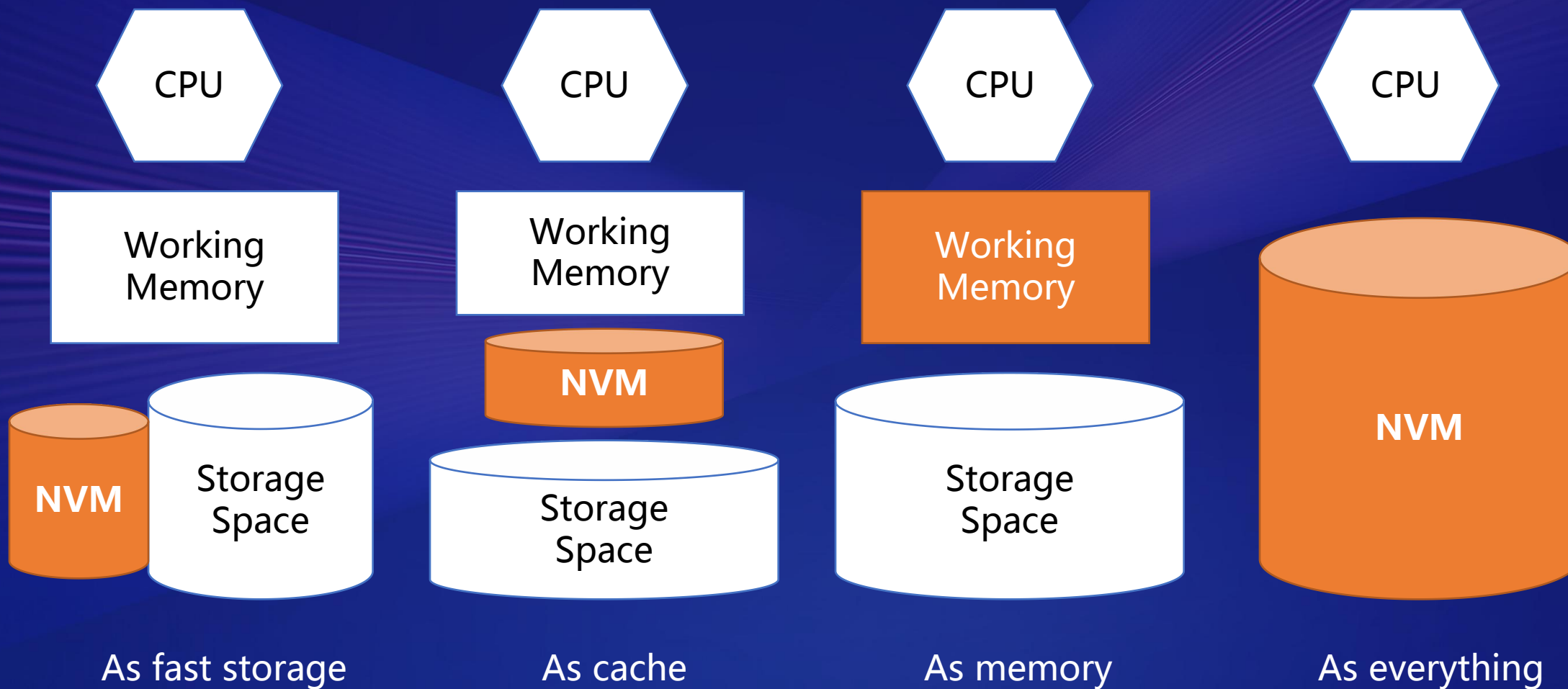
Source: <https://thememoryguy.com/intels-optane-dimm-price-model/>

存储介质在数据库系统中的角色

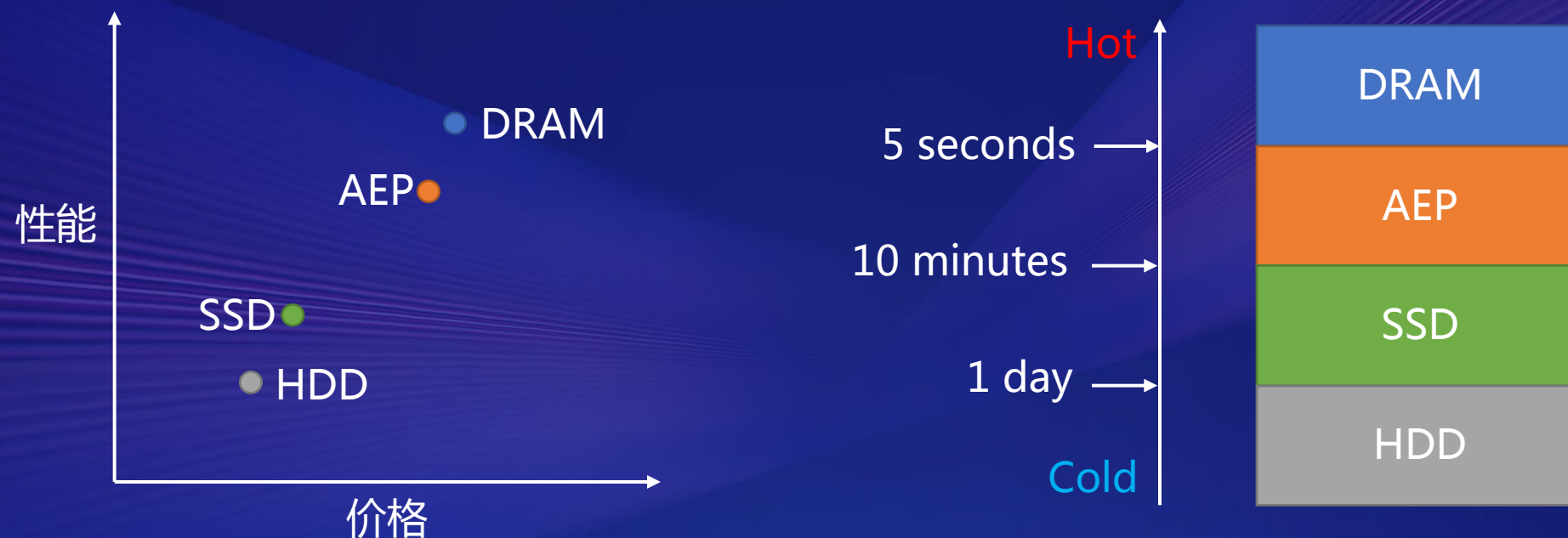
- Storage Space
 - 数据访问：读取和更新
 - 持久化：确保数据操作不丢失
- Working Space
 - 计算：即内存，完成控制、调度、查询处理的计算过程



不一样的选择

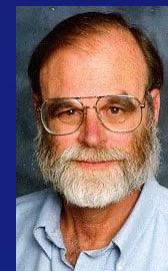


NVMM的定位取决于性价比

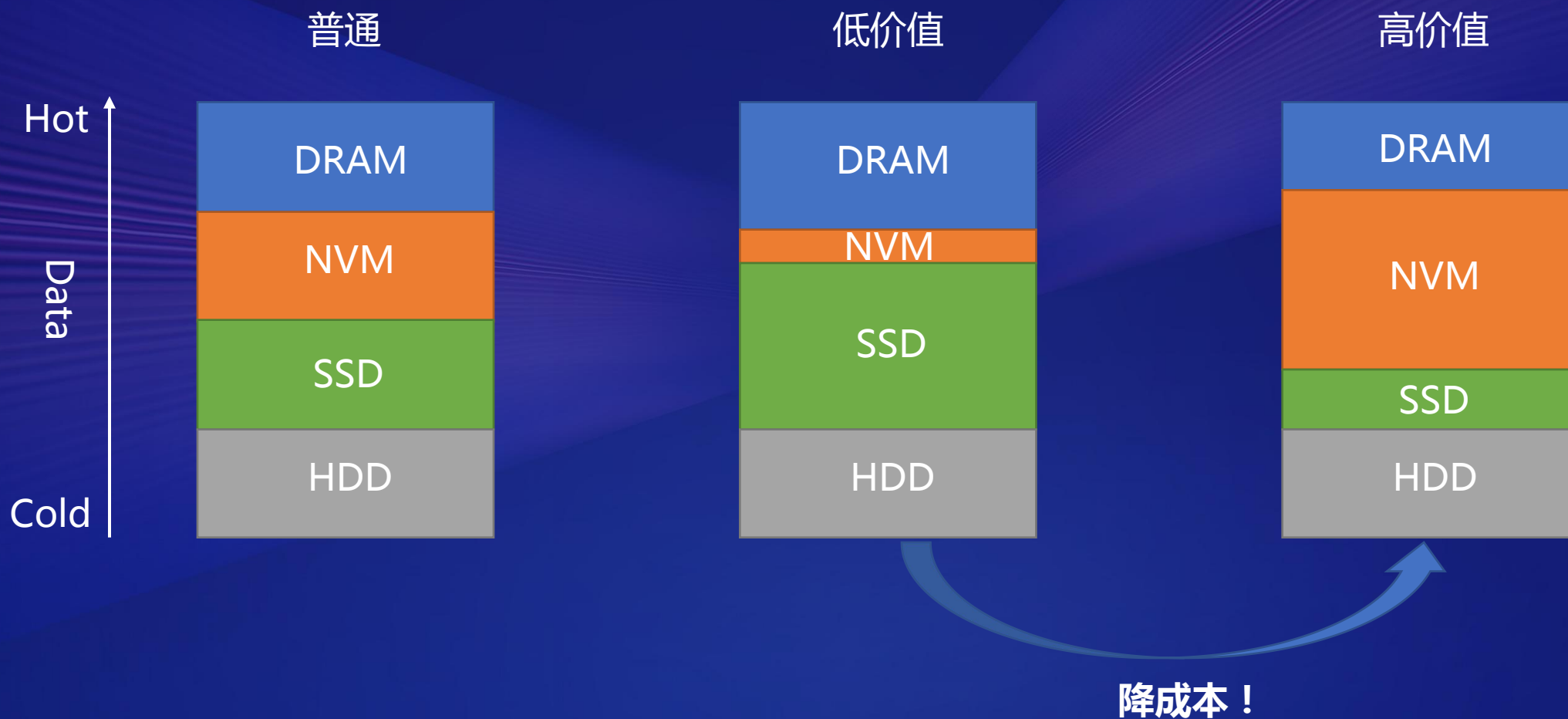


Five Minutes Rule: "The price of RAM to hold a 1-KB record was about equal to the (fractional) price of a disk drive required to access such a record every 5 minutes."

-Jim Gray

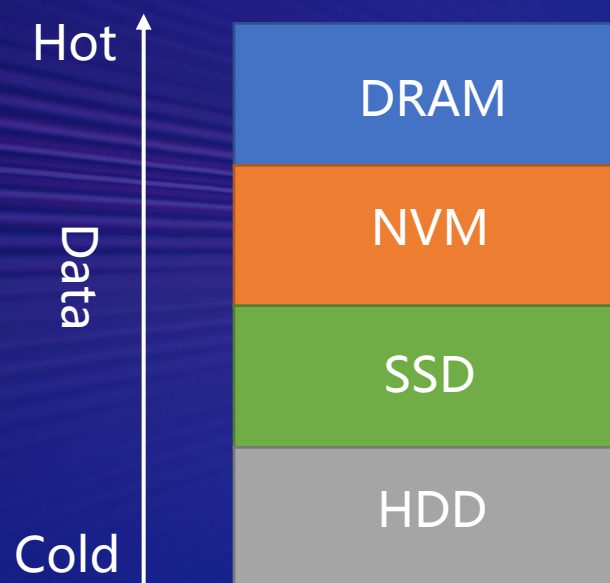


NVM的定位



NVM在数据库系统中的定位

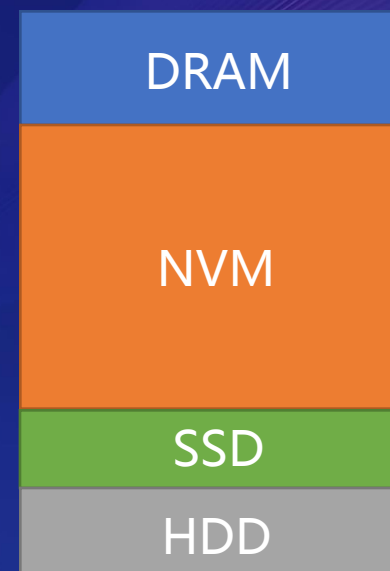
普通场景



As cache ...
Hybrid DBMS

扩展现有系统，
复杂度增加！

高价值场景

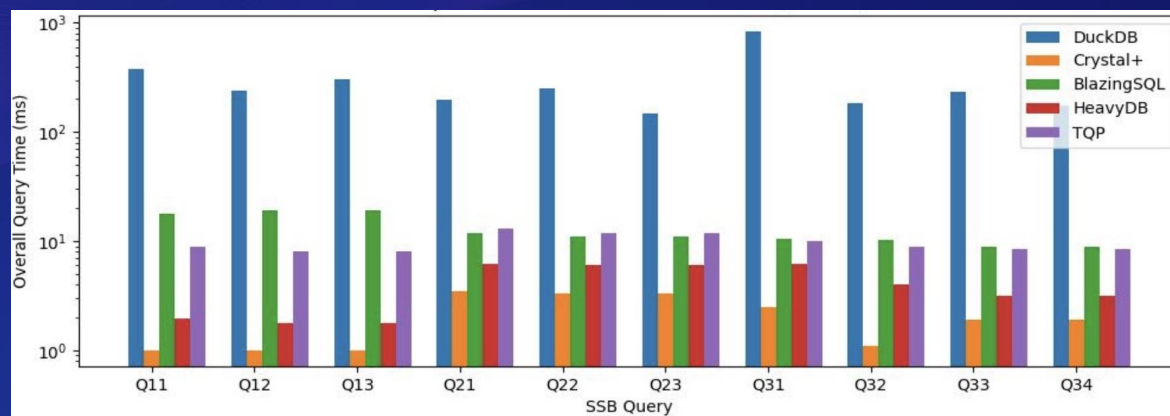
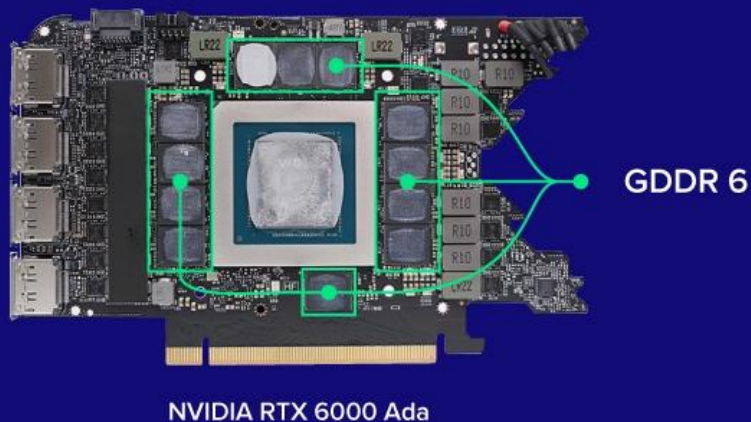


As everything
In-NVM DBMS

**OLTP数据库的
革新！**

GPU

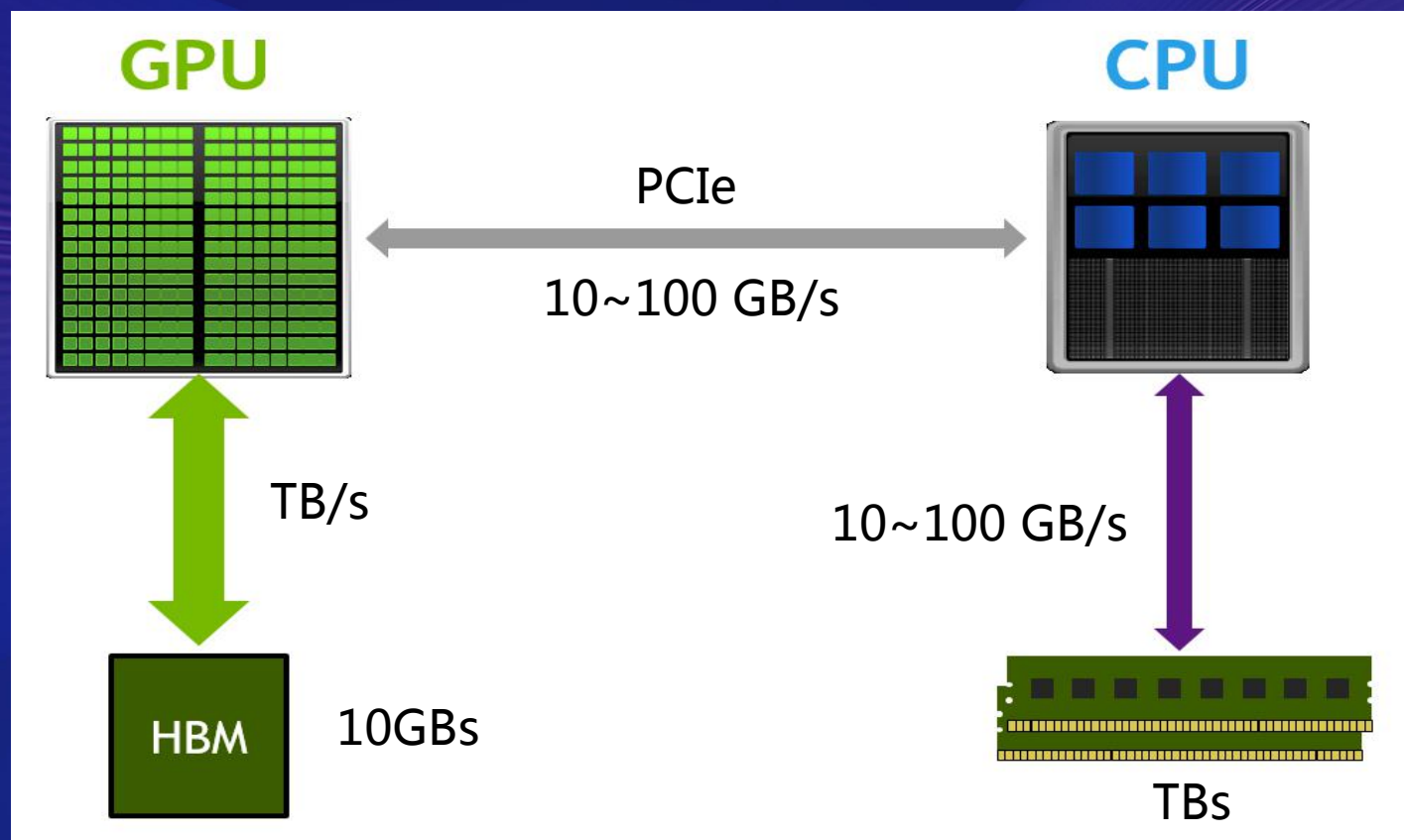
- 大规模并行 → 更便宜的算力
 - 还在持续提升：7nm→4nm→2nm
- 高带宽的显存：H100的HBM带宽达3.35TB/S
- 非常适合OLAP计算！



Sharma, Harshit, and Anmol Sharma. "A Comprehensive Overview of GPU Accelerated Databases." arXiv preprint arXiv:2406.13831 (2024).

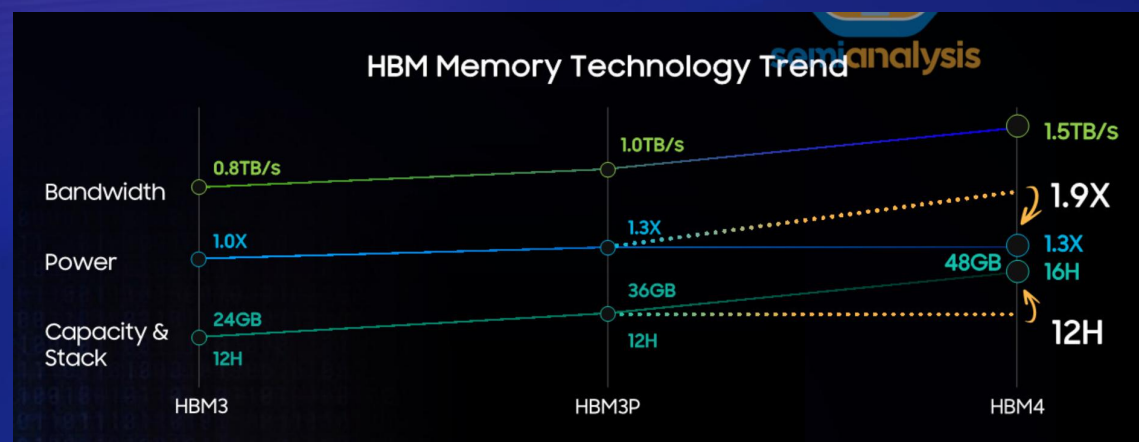
GPU OLAP性能受限于数据通道

- 并且，片外带宽落后于片内带宽是不变的趋势。



希望：HBM的进一步增长

- 如前所述，DRAM的增长缓解了I/O的压力
- 同理，HBM的增长会缓解GPU-DRAM通道的压力
 - 更多的数据可以缓存在HBM中，减少对DRAM的访问
- H100的HBM已经高达80GB。未来的HBM可达数百GB。



专用硬件

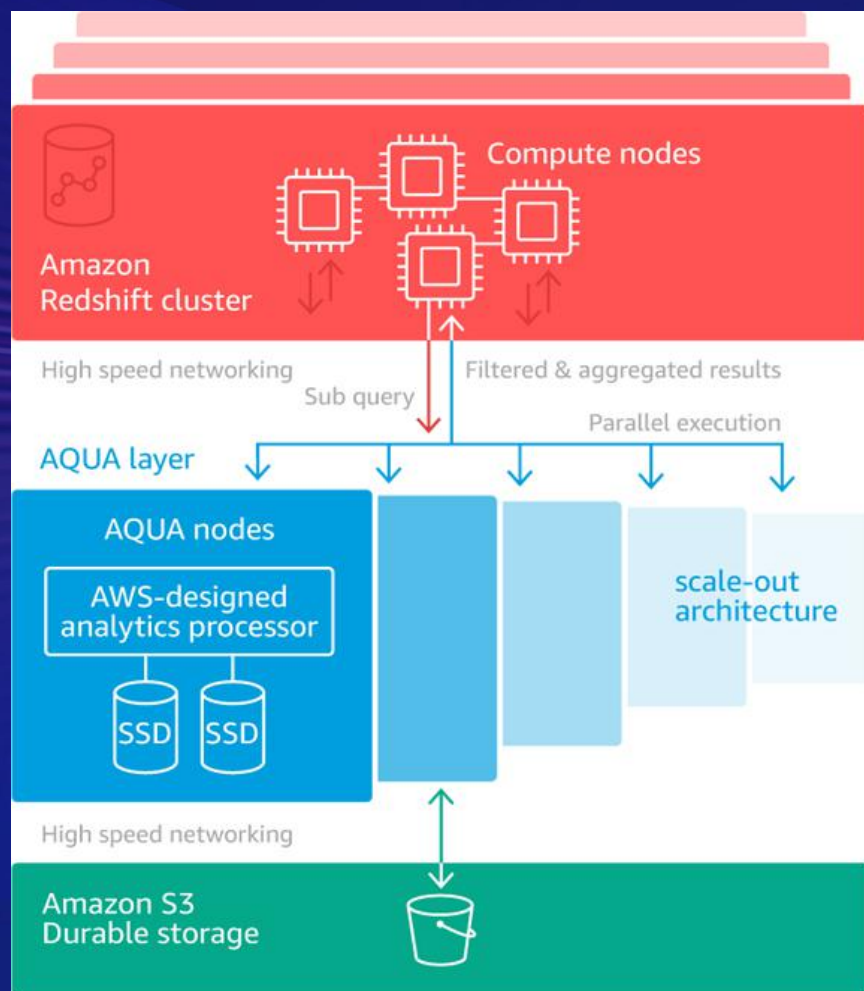


Project Corsica -- a New Zipline ASIC



KRYPTRO FPGA -- Allows Verification
Using Hardware Encryption

专用硬件案例：AWS AQUA (Advanced Query Accelerator)



“Within an AQUA node, there are multiple hardware modules. Custom silicon for compression and encryption, along with custom FPGA’ s (Field-programmable gate array) for analytics operations like scanning, filtering, and aggregation.”

“AQUA is designed to deliver up to 10X performance on queries that perform large scans, aggregates, and filtering with LIKE and SIMILAR_TO predicates. Over time we expect to add support for additional queries.”

专用硬件案例：阿里云PolarDB-X

• 用FPGA加速LSM-Tree的Compaction

FPGA-Accelerated Compactions for LSM-based Key-Value Store

Teng Zhang^{*,†}, Jianying Wang^{*}, Xuntao Cheng^{*}, Hao Xu^{*}, Nanlong Yu[†], Gui Huang^{*}, Tieying Zhang^{*},
 Dengcheng He^{*}, Feifei Li^{*}, Wei Cao^{*}, Zhongdong Huang[†], and Jianling Sun[†]

^{*}Alibaba Group

[†]Alibaba-Zhejiang University Joint Institute of Frontier Technologies, Zhejiang University
 {jason.zt, beilou.wjy, xuntao.cxt, hao.xh, qushan, tieying.zhang,
 dengcheng.hedc, lifeifei, mingsong.cw}@alibaba-inc.com
 {yunanlong, hzd, sunjl}@zju.edu.cn

Abstract

Log-Structured Merge Tree (LSM-tree) key-value (KV) stores have been widely deployed in the industry due to its high write efficiency and low costs as a tiered storage. To maintain such advantages, LSM-tree relies on a background compaction operation to merge data records or collect garbages for housekeeping purposes. In this work, we identify that slow compactions jeopardize the system performance due to unchecked oversized levels in the LSM-tree, and resource contentions for the CPU and the I/O. We further find that the rising I/O capabilities of the latest disk storage have pushed compactions to be bounded by CPUs when merging short KVs. This causes both query/transaction processing and background compactions to compete for the bottlenecked CPU resources extensively in an LSM-tree KV store.

In this paper, we propose to offload compactions to FPGAs

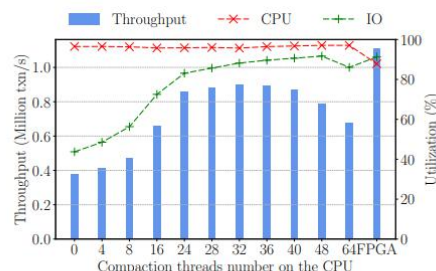


Figure 1: Impacts of added CPU threads for compactions on the overall system throughput.

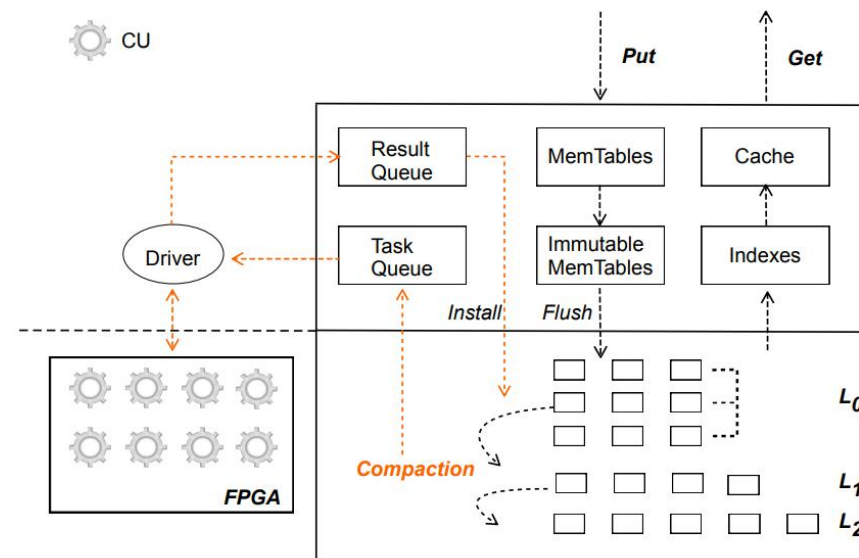


Figure 4: System Overview of the Storage Engine with the FPGA-offloading of Compactions.

专用硬件的局限性

- 开发、维护成本高
- 只能应用于局部计算
- 无法克服数据传输通道的瓶颈
- 能否应用于内核？比如并发控制、日志、查询引擎

总结

- 数据库系统性能瓶颈的变化：I/O $\rightarrow$ I/O & CPU
- 未来硬件可能带来的影响
 - 非易失内存（NVM）：OLTP数据库的演进；前提是NVM性价比提升。
 - GPU：OLAP数据库的再一次演进；前提是HBM增长。
 - 专用硬件：局限于RDS等规模化应用。
- 提高数据库对硬件的适应能力需要降低系统的复杂性

谢谢大家！

